



Actividad 60% 3ª evaluación backend, frontend, base de datos – 1º DAW

Sergio Ruiz Montesinos

Profesores: José Luis López, Esteban Castañer, Pedro Campos

31/06/2026

LoveCode

Contenido

1. Introducción.....	4
2. Descripción General del Sistema	4
2.1 Base de datos (MySQL)	4
2.2 Backend (Java).....	4
2.3 Frontend (HTML, CSS y JS).....	4
3. Bases de Datos	5
3.1 Estructura de la base de datos.....	5
3.2 Usuarios de la base de datos y permisos	7
3.3 Procedimientos almacenados	8
3.4 Triggers	10
3.5 Consultas SQL	11
3.6 Evidencias del funcionamiento de la base de datos en virtual box	14
4. Programación (Java)	14
4.1 Estructura del proyecto.....	14
4.2 Conexión a la base de datos mediante JDBC	14
4.3 Endpoints	15
4.4 Registro, login y operaciones principales.....	16
4.5 Evidencias de la conexión a la base de datos	17
4.6 Estructura del proyecto.....	18
5. Lenguajes de Marcas (Frontend).....	18
5.1 Paginas desarrolladas	19
5.2 Login	19
5.3 Registro con tecnologías.....	20
5.4 Perfiles, likes y matches	20
5.5 Gestión de sesión con localStorage	22
6. Entornos de Desarrollo (Git)	22
6.1 Estructura del repositorio.....	22
6.2 Ramas y commits	24

6.3 Pruebas unitarias	24
6.4 Test implementados	24
7. Sistemas Informáticos	25
7.1 Base de datos en máquina virtual.....	25
7.2 Backup de la base de datos por terminal	25
7.3 Script automático de backup	26
7.4 Backup manual.....	27
7.5 Evidencias del funcionamiento de la virtual box.....	29
8. Problemas Encontrados y Soluciones	29
8.1 Configuración de acceso remoto a MySQL	29
8.2 Credenciales en la clase conexionbd	29
8.3 Posicionamiento CSS en la página de inicio.....	30
8.4 Automatización del backup con cron	30
8.5 CORS en el backend.....	30
9. Conclusiones	30
9.1 Lo aprendido.....	30
10. Anexos.....	31
Base de datos	31
Scripts	32

1. Introducción

LoveCode es una aplicación web orientada a la conexión entre personas con intereses relacionados con la programación y la tecnología. El concepto se basa en el modelo de aplicaciones de perfiles, donde los usuarios pueden descubrir a otros, dar likes y, si el interés es mutuo, generar un match.

Esta memoria documenta el desarrollo completo del proyecto, que incluye la fase inicial con la estructura básica de la base de datos, la conexión java y el frontend estático, y la fase final con el backend, spring boot funcional, las API completas y el frontend dinámico conectado.

2. Descripción General del Sistema

2.1 Base de datos (MySQL)

Se utiliza MySQL como sistema gestor de base de datos. La base de datos LoveCode contiene la tabla de usuarios, tecnologías, likes, matches y usuario_tecnologia. Incluye los usuarios de acceso con sus permisos diferenciados tres procedimientos almacenados y un trigger automático de match.

2.2 Backend (Java)

El backend está desarrollado en Java. Gestiona el registro, login, consulta de perfiles, gestión de likes y matches. Se comunica con la base de datos a través de JDBC. El controlador lleva anotación `@CrossOrigin` para permitir peticiones desde el navegador.

2.3 Frontend (HTML, CSS y JS)

Interfaz web compuesta por siete paginas. Se comunica con el backend mediante la API Fetch de JavaScript enviando peticiones HTTP. El id del usuario autenticado se almacena en localStorage para mantener la sesión durante la navegación entre páginas.

3. Bases de Datos

3.1 Estructura de la base de datos

La base de datos se denomina LoveCode. A continuación, se describen las cinco tablas principales.

Tabla usuario almacena todos los datos de las personas registradas en la aplicación.

```
CREATE TABLE usuario (
    id_usuario INT PRIMARY KEY NOT NULL AUTO_INCREMENT,
    nombre varchar(100) DEFAULT NULL,
    email varchar(150) NOT NULL UNIQUE KEY,
    contrasena varchar(255) NOT NULL,
    descripcion text DEFAULT NULL,
    fecha_registro datetime DEFAULT current_timestamp(),
    ciudad varchar(100) DEFAULT NULL,
    estado_cuenta enum('activo','inactivo') DEFAULT 'activo',
    apellido varchar(255) DEFAULT NULL,
    direccion varchar(255) DEFAULT NULL,
    usuario varchar(255) DEFAULT NULL,
) ENGINE=InnoDB
```

Tabla: tecnología almacena el catálogo de tecnologías disponibles en la plataforma

```
CREATE TABLE tecnologia (
    id_tecnologia INT PRIMARY KEY NOT NULL AUTO_INCREMENT,
    nombre varchar(100) NOT NULL,
    categoria varchar(100) DEFAULT NULL,
    descripcion text DEFAULT NULL,
    fecha_creacion datetime DEFAULT current_timestamp(),
) ENGINE=InnoDB
```

Tabla usuario_tecnologia es la tabla intermedia que resuelve la relación muchos a muchos entre usuario y tecnología

```
CREATE TABLE usuario_tecnologia (
    id_usuario_tecnologia INT PRIMARY KEY NOT NULL AUTO_INCREMENT,
    id_usuario INT DEFAULT NULL,
    id_tecnologia INT DEFAULT NULL,
    nivel enum('basico','intermedio','avanzado') DEFAULT 'basico',
    fecha_asociacion datetime DEFAULT current_timestamp(),
    FOREIGN KEY (id_usuario) REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    FOREIGN KEY (id_tecnologia) REFERENCES tecnologia (id_tecnologia) ON DELETE
    CASCADE
) ENGINE=InnoDB
```

Tabla likes registra cada vez que un usuario da like a otro

```
CREATE TABLE likes (
    id_like INT AUTO_INCREMENT PRIMARY KEY,
    usuario_manda INT NOT NULL,
    usuario_recibe INT NOT NULL,
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (usuario_manda) REFERENCES usuario (id_usuario) ON DELETE
    CASCADE,
    FOREIGN KEY (usuario_recibe) REFERENCES usuario (id_usuario) ON DELETE
    CASCADE,
    UNIQUE KEY uq_like (usuario_manda, usuario_recibe)
) ENGINE = InnoDB;
```

Tabla matches se rellena automáticamente mediante el trigger cuando dos usuarios se han dado like mutuamente

```
CREATE TABLE matches (
  id_match INT AUTO_INCREMENT PRIMARY KEY,
  usuario_manda INT NOT NULL,
  usuario_recibe INT NOT NULL,
  fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (usuario_manda) REFERENCES usuario (id_usuario),
  FOREIGN KEY (usuario_recibe) REFERENCES usuario (id_usuario)
)ENGINE = InnoDB;
```

3.2 Usuarios de la base de datos y permisos

Se han creado tres usuarios en MySQL con niveles de acceso diferenciados según su rol dentro del sistema:

Código creación BBDD:

Usuario root (administrador)

Tiene todos los privilegios sobre la base de datos LoveCode. Es el usuario administrador del sistema.

```
CREATE USER 'root'@'localhost' IDENTIFIED BY '1234';
GRANT ALL PRIVILEGES ON LoveCode. * TO 'root'@'localhost';
FLUSH PRIVILEGES;
```

Usuario desarrollador

Tiene permisos de lectura, inserción, actualización y borrado. Es el usuario que utiliza el backend Java para operar con la base de datos.

Código creación BBDD

```
CREATE USER "desarrollador"@"localhost" IDENTIFIED BY "DevPass456";

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER ON
LoveCode. * TO 'desarrollador'@'localhost';

FLUSH PRIVILEGES;
```

Usuario lector

Solo tiene permisos de lectura (SELECT). Es un usuario de solo consulta, pensado para tareas de monitorización o reporting.

Código creación BBDD

```
CREATE USER 'lector'@'localhost' IDENTIFIED BY 'lectorpass';

GRANT SELECT ON LoveCode. * TO 'lector'@'localhost';

FLUSH PRIVILEGES;FLUSH PRIVILEGES;
```

3.3 Procedimientos almacenados

Procedimiento 1: Conteo de matches

Devuelve el número total de matches que tiene un usuario determinado.

```
DELIMITER //

CREATE PROCEDURE conteo_matches()

BEGIN

SELECT COUNT(*) AS total_matches

FROM matches;

END //

DELIMITER //
```


Procedimiento 2: Carga de datos de usuario

Devuelve todos los datos del perfil de un usuario a partir de su identificador.

DELIMITER //

CREATE PROCEDURE datos_usuario (IN p_id_usuario INT)

BEGIN

SELECT nombre, email, contrasena, descripcion, fecha_registro, ciudad,
estado_cuenta

FROM usuario

WHERE id_usuario = p_id_usuario;

END //

DELIMITER ;

Procedimiento 3: Borrado de usuario

Elimina un usuario de la base de datos a partir de su identificador.

DELIMITER //

CREATE PROCEDURE borrado_usuario (IN p_id_usuario INT)

BEGIN

DELETE FROM usuario WHERE id_usuario = p_id_usuario;

END //

DELIMITER ;

3.4 Triggers

Se ejecuta automáticamente cuando dos usuarios se dan like mutuamente y hacen match

```
DELIMITER //
```

```
CREATE TRIGGER trigger_match
```

```
AFTER INSERT ON likes
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF EXISTS (
```

```
SELECT 1 FROM likes
```

```
WHERE usuario_manda = NEW.usuario_recibe
```

```
AND usuario_recibe = NEW.usuario_manda )
```

```
THEN
```

```
INSERT INTO
```

```
matches (usuario_manda, usuario_recibe)
```

```
VALUES (NEW.usuario_manda, NEW.usuario_recibe);
```

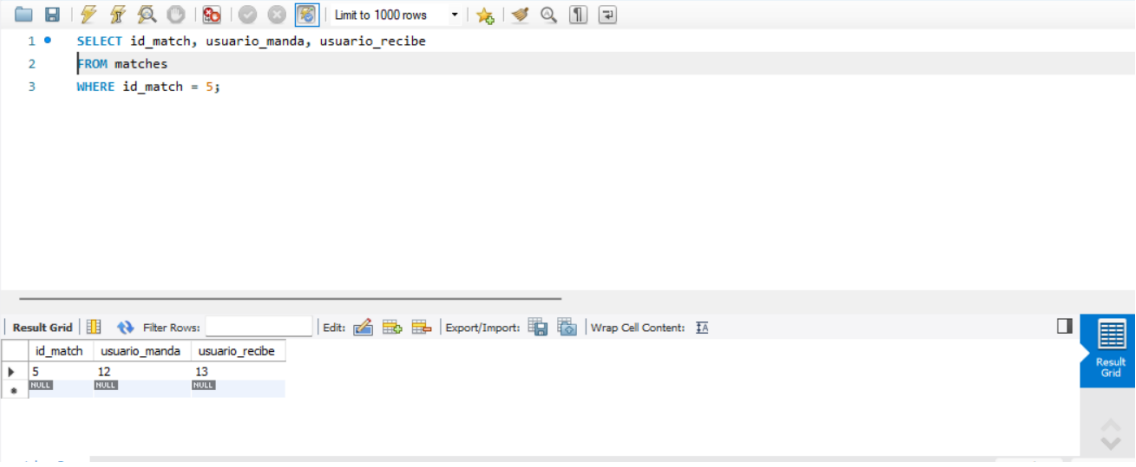
```
END IF;
```

```
END //
```

```
DELIMITER ;
```

3.5 Consultas SQL

Consulta la tabla matches y muestra únicamente el registro cuyo id_match sea 5



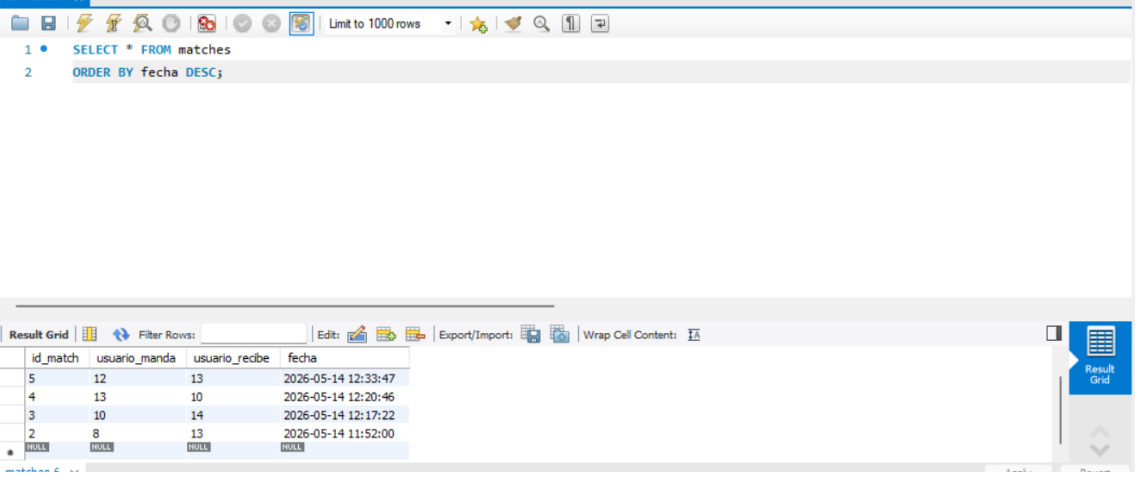
The screenshot shows a SQL query editor with the following query:

```
1 • SELECT id_match, usuario_manda, usuario_recibe
2 FROM matches
3 WHERE id_match = 5;
```

The result grid below the query shows one row of data:

id_match	usuario_manda	usuario_recibe
5	12	13

Muestra todos los matches que hay



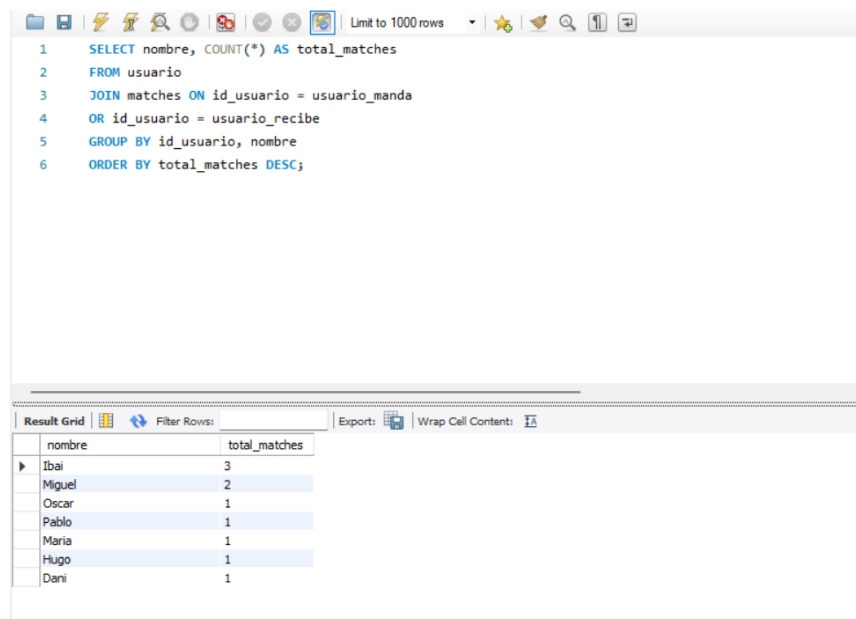
The screenshot shows a SQL query editor with the following query:

```
1 • SELECT * FROM matches
2 ORDER BY fecha DESC;
```

The result grid below the query shows five rows of data:

id_match	usuario_manda	usuario_recibe	fecha
5	12	13	2026-05-14 12:33:47
4	13	10	2026-05-14 12:20:46
3	10	14	2026-05-14 12:17:22
2	8	13	2026-05-14 11:52:00
1	NULL	NULL	NULL

Contar cuantos matches tiene cada usuario



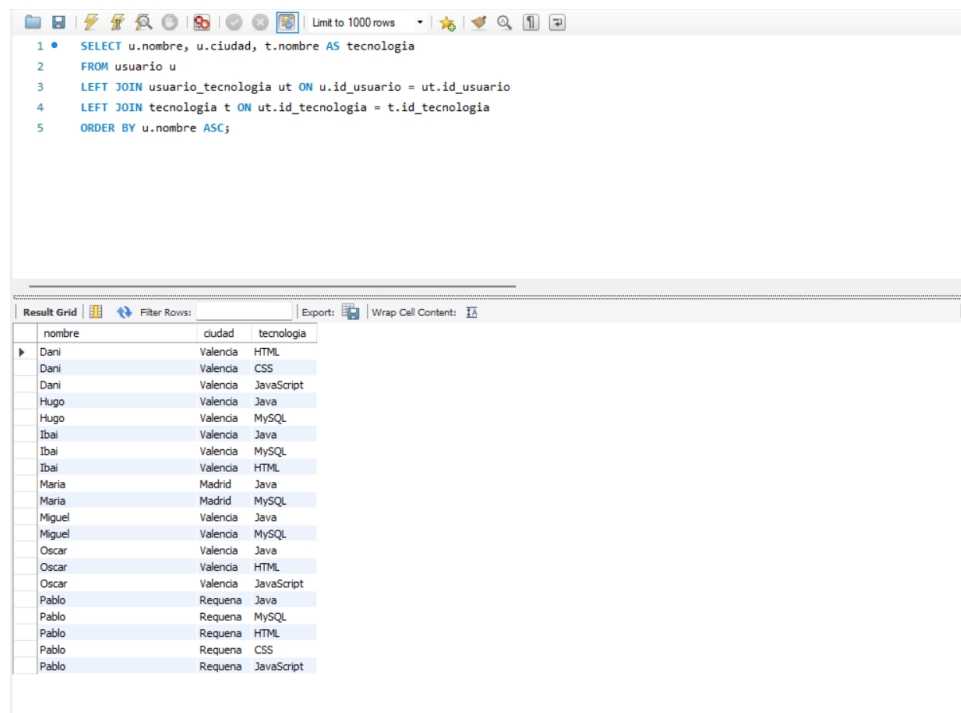
The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1 SELECT nombre, COUNT(*) AS total_matches
2 FROM usuario
3 JOIN matches ON id_usuario = usuario_manda
4 OR id_usuario = usuario_recibe
5 GROUP BY id_usuario, nombre
6 ORDER BY total_matches DESC;
```

Below the query editor is the 'Result Grid' tab, which displays the results of the query. The grid has two columns: 'nombre' and 'total_matches'. The results are as follows:

nombre	total_matches
Ibai	3
Miguel	2
Oscar	1
Pablo	1
Maria	1
Hugo	1
Dani	1

Ver todos los perfiles con sus tecnologías



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1 SELECT u.nombre, u.ciudad, t.nombre AS tecnologia
2 FROM usuario u
3 LEFT JOIN usuario_tecnologia ut ON u.id_usuario = ut.id_usuario
4 LEFT JOIN tecnologia t ON ut.id_tecnologia = t.id_tecnologia
5 ORDER BY u.nombre ASC;
```

Below the query editor is the 'Result Grid' tab, which displays the results of the query. The grid has three columns: 'nombre', 'ciudad', and 'tecnologia'. The results are as follows:

nombre	ciudad	tecnologia
Dani	Valencia	HTML
Dani	Valencia	CSS
Dani	Valencia	JavaScript
Hugo	Valencia	Java
Hugo	Valencia	MySQL
Ibai	Valencia	Java
Ibai	Valencia	MySQL
Ibai	Valencia	HTML
Maria	Madrid	Java
Maria	Madrid	MySQL
Miguel	Valencia	Java
Miguel	Valencia	MySQL
Oscar	Valencia	Java
Oscar	Valencia	HTML
Oscar	Valencia	JavaScript
Pablo	Requena	Java
Pablo	Requena	MySQL
Pablo	Requena	HTML
Pablo	Requena	CSS
Pablo	Requena	JavaScript

Contar cuantos likes tiene cada usuario

```

1 • SELECT u.nombre, COUNT(l.id_like) AS likes_dados
2 FROM usuario u
3 LEFT JOIN likes l ON u.id_usuario = l.usuario_manda
4 GROUP BY u.id_usuario, u.nombre
5 ORDER BY likes_dados ASC;

```

Result Grid		Filter Rows:	Export:	Wrap Cell C	Result Grid
nombre	likes_dados				
▶ Maria	1				
Dani	1				
Pablo	2				
Hugo	2				
Miguel	2				
Oscar	2				
Ibai	4				

Cuántas tecnologías tiene cada usuario

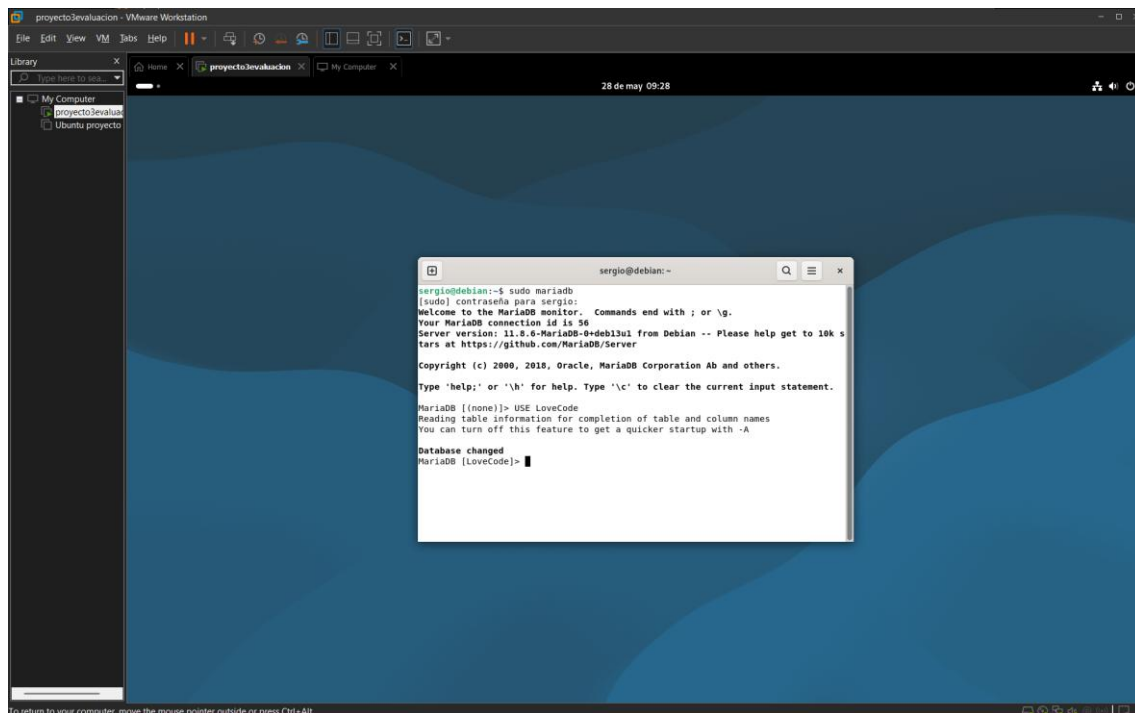
```

1 • SELECT u.nombre, COUNT(ut.id_tecnologia) AS num_tecnologias
2 FROM usuario u
3 LEFT JOIN usuario_tecnologia ut ON u.id_usuario = ut.id_usuario
4 GROUP BY u.id_usuario, u.nombre
5 ORDER BY num_tecnologias ASC;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
nombre	num_tecnologias			
▶ Maria	2			
Hugo	2			
Miguel	2			
Ibai	3			
Oscar	3			
Dani	3			
Pablo	5			

3.6 Evidencias del funcionamiento de la base de datos en virtual box



4. Programación (Java)

4.1 Estructura del proyecto

El proyecto Java está organizado con 4 clases principales:

- `conexionbd.java` - Gestiona la conexión con la base de datos mediante JDBC.
- `Main.java` – clase principal que arranca la aplicación Spring Boot
- `usuariocontrollerDAO.java` – controlador que define los endpoints de la API
- `usuarioDAO.java` – capa de acceso a datos con toda la lógica SQL

4.2 Conexión a la base de datos mediante JDBC

La clase `conexionbd` centraliza los parámetros de conexión y expone el método estático `getConnection()` para obtener una conexión activa. Los datos de configuración son:

URL: *jdbc:mysql://192.168.220.129:3306/LoveCode*

Usuario: *sergio\%*

Contraseña: *1234*

```

public static Connection getConnection() {
    Connection connection = null;

    try {
        connection = DriverManager.getConnection(URL, USER, PASSWORD);

        System.out.println("Conexion exitosa a la base de datos");
    } catch (SQLException e) {
        System.out.println("Error de conexion: " + e.getMessage());
    }

    return connection;
}

```

4.3 Endpoints

El controlador usa las anotaciones `@RestController`, `@RequestMapping` y `@CrossOrigin` para permitir peticiones desde el navegador. Los endpoints disponibles son

Método	Endpoint	Descripción
Post	/api/registro	Registro de nuevo usuario con tecnologías
Post	/api/login	Devuelve id y nombre del usuario
Get	/api/perfiles/{id}	Lista de perfiles (excluye al usuario actual)
Get	/api/tecnologías	Lista de tecnologías disponibles
Get	/api/usuario/{id}	Datos del perfil propio
Post	/api/like	Insertar un like entre dos usuarios
Get	/api/likes/{id}	Usuarios a los que se ha dado like
Post	/api/match	Insertar un match entre dos usuarios
Get	/api/matches/{id}	Lista de matches del usuario
Get	/api/matches/total/{id}	Contador de matches

4.4 Registro, login y operaciones principales

Registro

El método registro recibe un Map con los datos del formulario, ejecuta un INSERT en la tabla usuario obteniendo la clave generada, y si el usuario ha seleccionado tecnologías las inserta en usuario_tecnologia. Devuelve success=true o un mensaje de error

Login

El método login hace un SELECT filtrando por email y contraseña. Si encuentra resultado devuelve success=true junto con el id y nombre del usuario, que el frontend guarda en localStorage para mantener la sesión

Perfiles y tecnologías

El método perfilesPorId devuelve todos los usuarios menos el actual usando un LEFT JOIN con usuario_tecnologia y tecnologia. Las tecnologías se concatenan con GROUP_CONCAT para que el frontend las muestre como etiquetas individuales.

Likes, matches y notificaciones

Los métodos darLike y darMatch insertan registros en sus respectivas tablas. El endpoint /api/matches/total/{id} permite al frontend consultar periódicamente si han aparecido nuevos matches para mostrar una notificación emergente.

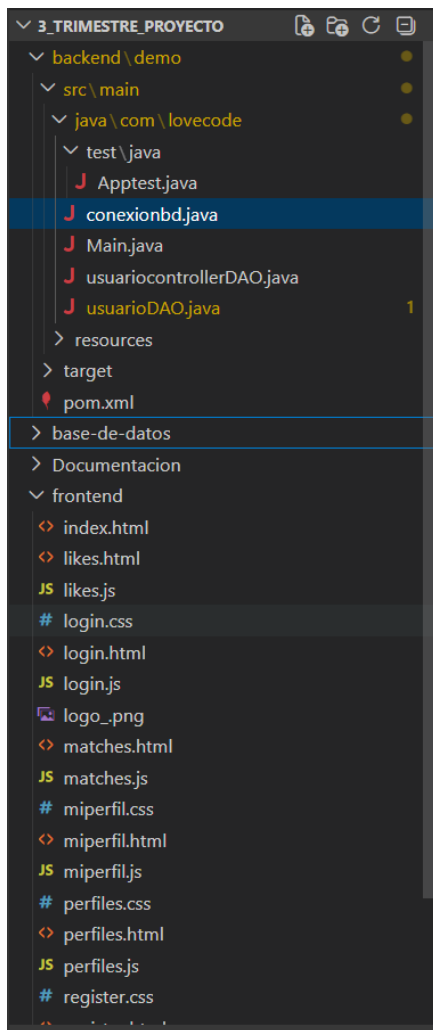
4.5 Evidencias de la conexión a la base de datos

```
backend > demo > src > main > java > com > lovecode > J conexionbd.java > Language Support for Java(TM)
1 package com.lovecode;
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.SQLException;
5
6 public class conexionbd {
7
8     private static final String URL = "jdbc:mysql://192.168.220.129:3306/LoveC
9     private static final String USER = "sergio";
10    private static final String PASSWORD = "1234";
11
12    public static Connection getConnection() {
13        Connection connection = null;
14
15        try {
16            connection = DriverManager.getConnection(URL, USER, PASSWORD);
17            System.out.println("Conexión exitosa a la base de datos");
18        } catch (SQLException e) {
19            System.out.println("Error de conexión: " + e.getMessage());
20        }
21
22        return connection;
23    }
24 }
25
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\sergi\Desktop\backend\backend\backend\3_trimestre_proyecto> & 'C:\Users\sergi\
\vscod\extensions\redhat.java-1.54.0-win32-x64\jre\21.0.10-win32-x86_64\bin\java.exe' '@C:\Users\sergi\AppData\Local\Temp\cp_2j2ln9s0td193yctrduits.args' 'com.lovecode.Main'
ost],[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-05-30T16:02:13.562+02:00 INFO 24016 --- [nio-8080-exec-1] o.s.web.servlet.Dispatcher
Servlet : Initializing Servlet 'dispatcherServlet'
2026-05-30T16:02:13.563+02:00 INFO 24016 --- [nio-8080-exec-1] o.s.web.servlet.Dispatcher
Servlet : Completed initialization in 1 ms
Conexión exitosa a la base de datos
```

4.6 Estructura del proyecto



5. Lenguajes de Marcas (Frontend)

El frontend está compuesto por siete paginas HTML con sus ficheros JavaScript y CSS asociados. En la actividad del 15% las páginas eran estáticas, en la del 60% se ha incorporado JavaScript con llamadas a la API REST del backend

5.1 Paginas desarrolladas

Pagina	Js asociado	Css asociado	Descripción
Index.html		Style.css	Inicio: logo, eslogan y botones de acceso
Login.html	Login.js	Login.css	Formulario de inicio de sesión
Register.html	Register.js	Register.css	Formulario de registro con selección de tecnologías
Perfiles.html	Perfiles.js	Perfiles.css	Vista de perfiles con botones like y match
Miperfil.html	Miperfil.js	Miperfil.css	Perfil propio del usuario autenticado
Likes.html	Likes.js	Likes.css	Lista de usuario a los que se ha dado like
Matches.html	Matches.js	Matches.css	Lista de matches del usuario

5.2 Login

Login.js captura el email y la contraseña del formulario y hace un POST a /api/login. Si la respuesta tiene success=true guarda el id del usuario en localStorage y redirige a perfiles.html. Si las credenciales son incorrectas muestra una alerta.

```

10
11     fetch("http://localhost:8080/api/login", {
12         method: "POST",
13         headers: {"Content-Type": "application/json"},
14         body: JSON.stringify({
15             email: email,
16             contraseña: password
17         })
18     })
19     .then(response => response.json())
20     .then(data => {
21         if (data.success) {
22             localStorage.setItem("id_usuario", data.id);
23             alert("Inicio de sesión exitoso");
24             window.location.href = "perfiles.html";
25         } else {
26             alert("Credenciales incorrectas");
27         }
28     })
29     .catch(error => {
30         alert("Error al iniciar sesión");
31     });
32 });

```

5.3 Registro con tecnologías

Al cargar la página, register.js hace un GET a /api/tecnologías y genera dinámicamente un checkbox por cada tecnología disponible. Al enviar, recoge todos los campos del formulario más los ids de los checkboxes marcados y hace un POST a /api/registro.

```

1 window.onload = function() {
2   fetch("http://localhost:8080/api/tecnologias")
3   .then(response => response.json())
4   .then(tecnologias => {
5     const div = document.getElementById("tecnologias");
6     tecnologias.forEach(tech => {
7       const label = document.createElement("label");
8       label.innerHTML = `<input type="checkbox" value="${tech.id}"> ${tech.nombre}`;
9       div.appendChild(label);
10    });
11  })
12  .catch(error => {
13    console.log("Error cargando tecnologias");
14  });
15 }

```

```

fetch("http://localhost:8080/api/registro", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({
    nombre: nombre,
    apellido: apellido,
    ciudad: ciudad,
    direccion: direccion,
    usuario: usuario,
    email: email,
    contrasena: contrasena,
    descripcion: descripcion,
    tecnologias: tecnologiasSeleccionadas
  })
})
.then(response => response.json())
.then(data => {
  if (data.success) {
    alert("Registro exitoso");
    window.location.href = "login.html";
  } else {
    alert("Error al registrarse");
  }
})
.catch(error => {
  alert("No se puede conectar con el servidor");
});

```

5.4 Perfiles, likes y matches

Perfiles.js carga los perfiles del resto de usuarios con GET /api/perfiles/{id}. Para cada uno genera una tarjeta con nombre, ciudad, descripción y tecnologías como etiquetas. Cada tarjeta tiene dos botones: like (POST /api/like) y match (POST /api/match).

```

setInterval(function() {
  const id = localStorage.getItem("id_usuario");
  fetch("http://localhost:8080/api/matches/total/" + id)
    .then(response => response.json())
    .then(data => {
      if (matchesAnteriores === -1) {
        matchesAnteriores = data.total;
        return;
      }
      if (data.total > matchesAnteriores) {
        mostrarNotificacion("¡Tienes un nuevo match!");
      }
      matchesAnteriores = data.total;
    });
}, 5000);

```

```

window.onload = function() {
  const id = localStorage.getItem("id_usuario");
  fetch("http://localhost:8080/api/perfiles/" + id)
    .then(response => response.json())
    .then(usuario => {
      const contenedor = document.getElementById("contenedor");
      usuario.forEach(usuario => {
        const div = document.createElement("div");
        div.className = "container";
        div.innerHTML = `
          <h3>${usuario.nombre}</h3>
          <p>${usuario.ciudad || "Sin ciudad"}</p>
          <p>${usuario.descripcion || "Sin descripción"}</p>
          <div class="tecnologias">${usuario.tecnologias ? usuario.tecnologias.split(",").map(t => `<span>${t}</span>`).join(" ") : ""}</div>
          <div class="botones">
            <button onclick="match(${usuario.id})">Match</button>
            <button onclick="like(${usuario.id})">Like</button>
          </div>
        `;
        contenedor.appendChild(div);
      });
    })
    .catch(error => {
      alert("No se puede conectar con el servidor");
    });
}

```

```

function like(idRecibe) {
  const idManda = localStorage.getItem("id_usuario");
  fetch("http://localhost:8080/api/like", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      usuarioManda: parseInt(idManda),
      usuarioRecibe: idRecibe
    })
  })
    .then(response => response.json())
    .then(data => {
      if (data.success) {
        alert("Like dado!");
      }
    })
    .catch(error => {
      alert("Error al dar like");
    });
}

```

```
function match(idRecibe) {
  const idManda = localStorage.getItem("id_usuario");
  fetch("http://localhost:8080/api/match", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      usuarioManda: parseInt(idManda),
      usuarioRecibe: idRecibe
    })
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      alert("Match!");
    }
  })
  .catch(error => {
    alert("Error al hacer match");
  });
}
```

5.5 Gestión de sesión con localStorage

Al hacer login se guarda el id del usuario en localStorage. Todas las paginas protegidas leen ese id al cargar (window.onload) para hacer sus peticiones al backend. La función cerrarSesion elimina el id y redirige al inicio

6. Entornos de Desarrollo (Git)

Se ha creado un repositorio Git para gestionar el control de versiones del proyecto. El repositorio contiene los tres componentes principales del sistema organizados en carpetas.

6.1 Estructura del repositorio

LoveCode/

Documentación/

Memoria tecnica

base-de-datos/

BaseDeDatos_proyecto_3evaluacion.sql

Backup_lovecode.sql

Diagrama ER

backend/

conexionbd.java

Main.java

usuarioControllerDAO.java

usuarioDAO.java

frontend/

Index.html

Login.html

Register.html

Perfiles.html

Miperfil.html

Likes.html

Matches.html

Login.js

Register.js

Perfiles.js

Miperfil.js

Likes.js

Matches.js

Style.css

Login.css

Register.css

Perfiles.css

Miperfil.css

Likes.css

Matches.css

README.md

6.2 Ramas y commits

Se han utilizado 5 ramas durante el desarrollo del proyecto. Los commits se han realizado de forma incremental por componente, con mensajes como “actualización del front”, “diagrama BaseDeDatos” o “memoria técnica”. Se han realizado al menos un pull request con merge hacia la rama main.

6.3 Pruebas unitarias

El funcionamiento de las pruebas unitarias es verificar que los endpoints más importantes como login, registro y like funcionan correctamente tanto en pruebas de éxito y de error

6.4 Test implementados

Se han implementado 6 tests organizados en 3 grupos

Test	Descripción	Resultado esperado
loginCorrecto	Simula una respuesta exitosa de login con nombre de usuario	Success = true nombre = “sergio”
loginIncorrecto	Simula una respuesta fallida de login con credenciales incorrectas	Success = false
registroCorrecto	Simula una respuesta exitosa del registro de nuevo usuario	Success = true
registroIncorrecto	Simula una respuesta fallida del registro con datos incorrectos	Success = false
likeCorrecto	Simula una respuesta exitosa al dar like al usuario	Success = true
likeIncorrecto	Simula una respuesta fallida al intentar dar like	Success = false

7. Sistemas Informáticos

7.1 Base de datos en máquina virtual

La base de datos que utilizo es mariadb está instalada y funcionando en una máquina virtual con sistema operativo Debian. La dirección IP de la máquina virtual es 192.168.220.129, que es la que se utiliza en la cadena de conexión JDBC del backend Java.

Se ha verificado que la base de datos es accesible desde la máquina anfitriona a través del puerto.

7.2 Backup de la base de datos por terminal

Se ha realizado una copia de seguridad completa de la base de datos utilizando el comando mysqldump.

Comando utilizado para generar el backup:

```
mysqldump -u root -p LoveCode > backup_lovecode.sql
```

El fichero backup_LoveCode.sql ha sido subido al repositorio Git dentro de la carpeta database/, junto con el script principal de creación de la base de datos.

7.3 Script automático de backup

El script de backup esta desarrollado en bash y se ejecuta desde la terminal de la maquina virtual Debian. Su función es generar una copia de seguridad completa de la base de datos LoveCode mediante el comando mysqldump

\$mysqlDump = "C:\Program Files\MySQL\MySQL Server 9.6" **# guarda la ruta exacta del ordenador donde está instalado el programa**

\$servidor = "192.168.220.129" **# dirección ip de la maquina virtual donde esta MariaDB**

\$usuario = "sergio" **# el usuario de acceso que usara el script para entrar a la base de datos**

\$password = "1234" **# contraseña del usuario con el que entras a la base de datos**

\$baseDatos = "LoveCode" **# nombre de la base de datos**

\$carpetaBackup = "C:\Users\sergi\Documents\dumps" **# ruta de la carpeta en mi maquina anfitriona donde se va a guardar el archivo .sql**

if (!(Test-Path \$carpetaBackup)) { **# comprueba si la carpeta de destino existe en el disco duro**

New-Item -ItemType Directory -Path \$carpetaBackup **# si la carpeta no existe este comando lo crea en mi sistema**

}

\$fecha = Get-Date -Format "2026-05-07_12-43" **# obtiene la fecha y hora exacta del sistema en el momento en que se ejecuta el script**

\$archivoBackup = "\$carpetaBackup\backup_{\$baseDatos}_\$fecha.sql" **# junta todas las variables anteriores para construir la ruta final del archivo de texto**

& \$mysqlDump -h \$servidor -u \$usuario "-p\$password" \$baseDatos > \$archivoBackup
comando para hacer el backup

```
if (Test-Path $archivoBackup) { # revisa si tras la ejecución el archivo .sql se ha
creado
```

```
    Write-Host "Backup creado correctamente en: $archivoBackup" # mensaje de
éxito que sale por pantalla
```

```
} else {
```

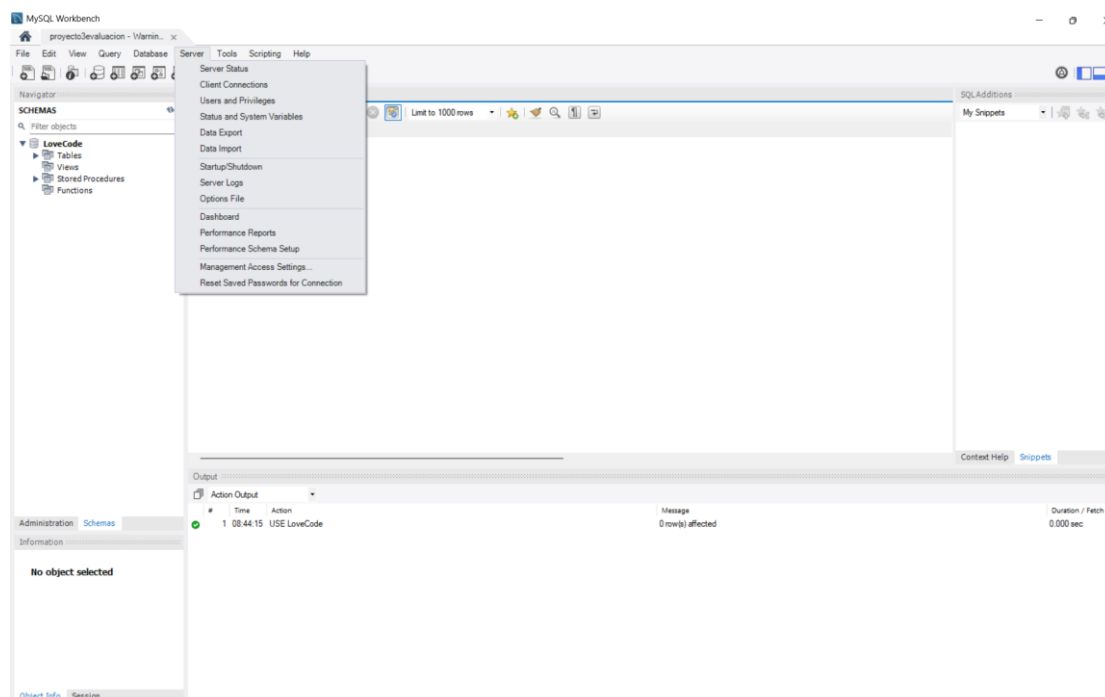
```
    Write-Host "ERROR: No se ha podido crear el backup" # mensaje de error
que sale por pantalla
```

```
}
```

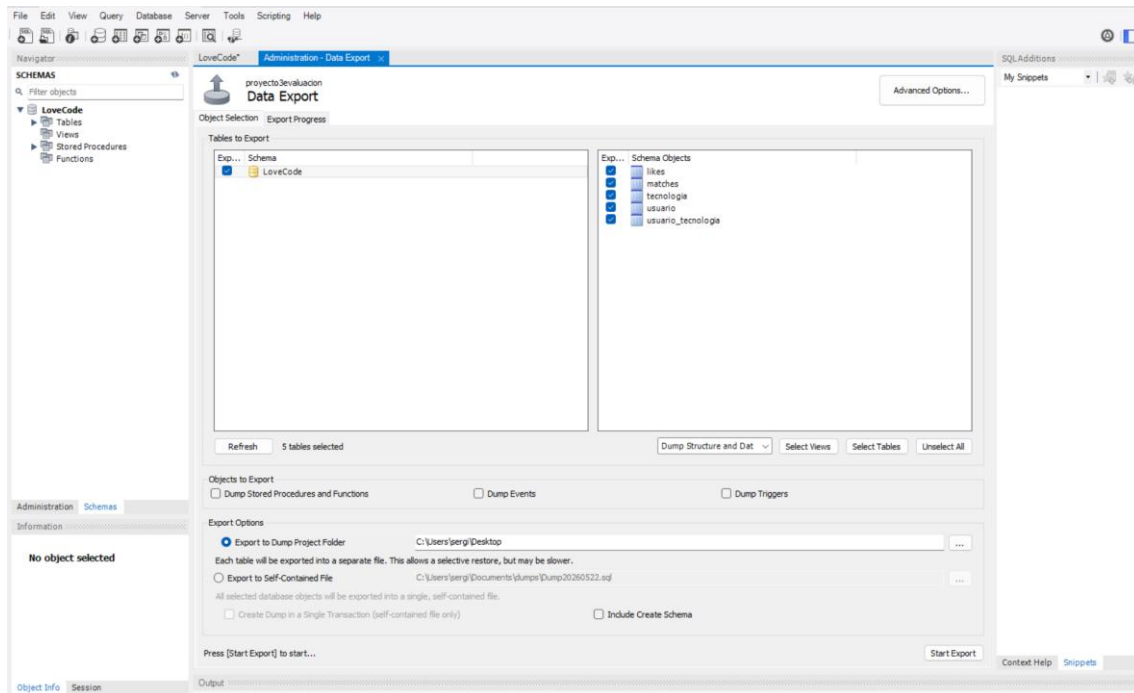
E intentado automatizarlo con cron pero no pude hacerlo funcionar debido a un error de compatibilidad entre la versión de MySQL Workbench instalada en el equipo local y la versión de MariaDB que corría en la máquina virtual.

7.4 Backup manual

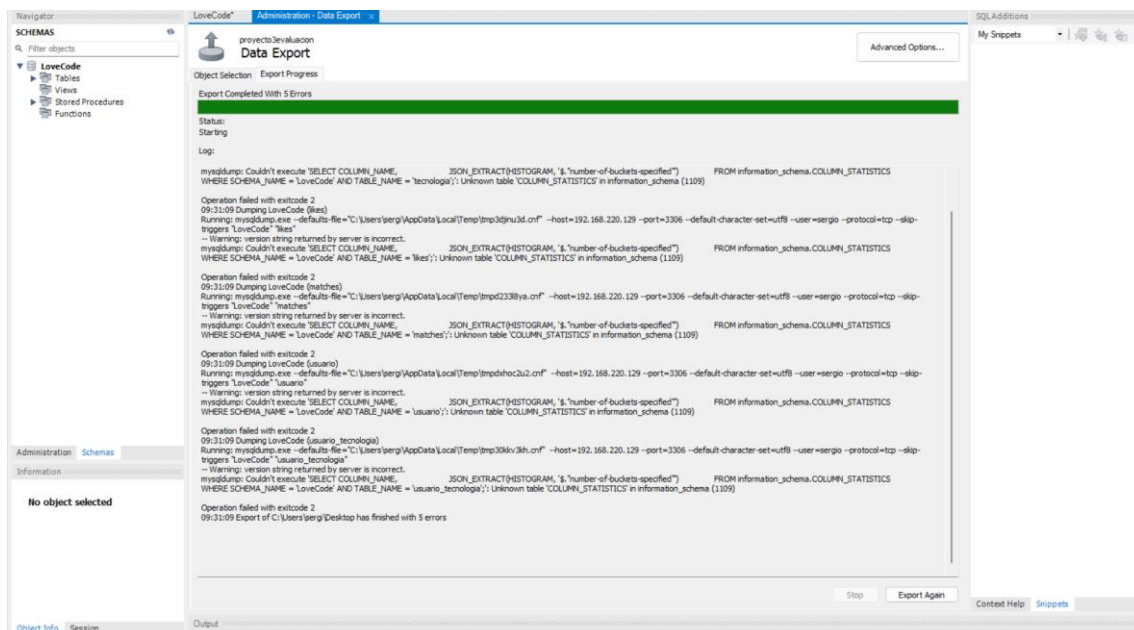
Para realizar la copia de seguridad se utilizo la herramienta data export de MySQL Workbench. Desde server data export.



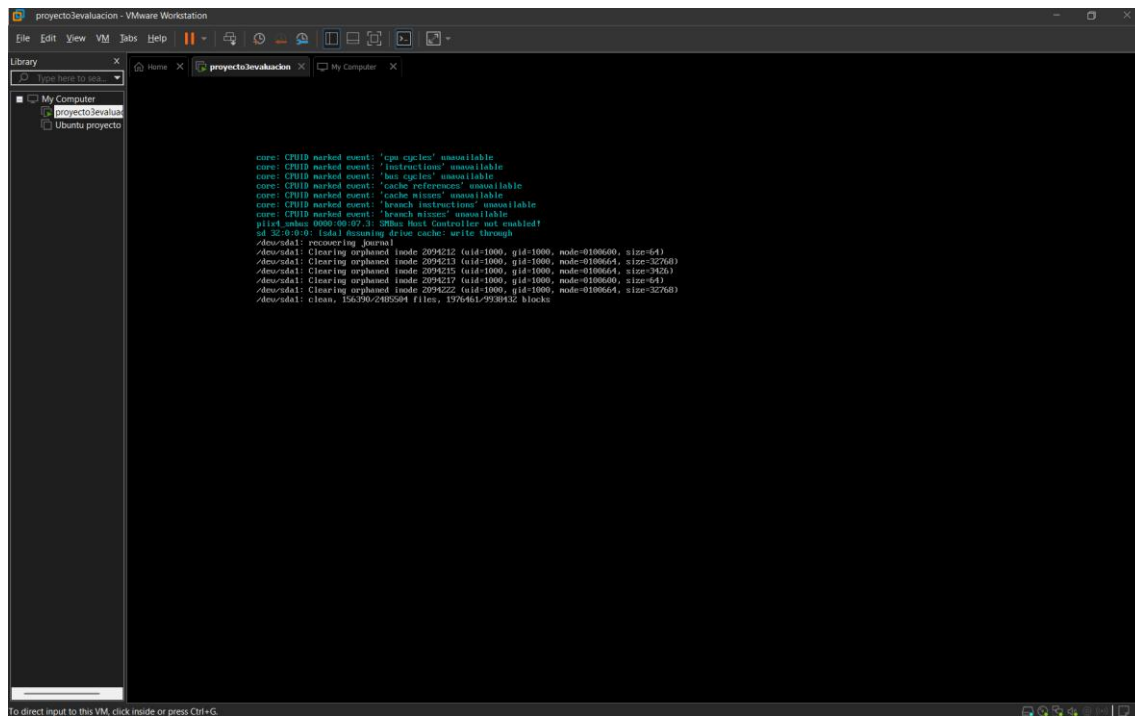
Se selecciona la base de datos LoveCode con todas sus tablas y se exporto a un archivo .sql.



Durante el proceso aparecieron errores de compatibilidad entre la versión de MySQL Workbench y MariaDB, y el archivo no se pudo generar



7.5 Evidencias del funcionamiento de la virtual box



8. Problemas Encontrados y Soluciones

8.1 Configuración de acceso remoto a MySQL

Al intentar conectar el backend Java con la base de datos en la máquina virtual surgieron errores de acceso. El problema era que MySQL por defecto solo acepta conexiones desde localhost. Se solucionó configurando el usuario desarrollador con el host '%' para permitir conexiones desde cualquier dirección IP, y verificando que el puerto 3306 estaba abierto en el firewall de la máquina virtual.

8.2 Credenciales en la clase conexionbd

El nombre de usuario MySQL contenía caracteres especiales que causaban problemas al pasarlo como cadena de texto en Java. Se revisó y corrigió la definición de las constantes USER y PASSWORD en la clase conexionbd.

8.3 Posicionamiento CSS en la página de inicio

El uso de `position: fixed` en los títulos h1 y h2 de la página de inicio causaba solapamientos con otros elementos en determinados tamaños de ventana. Se ajustaron los valores de `top` y `z-index` para que los elementos no interfirieran entre sí.

8.4 Automatización del backup con cron

Se intento programar la ejecución automática del backup con cron en Debian, pero surgieron errores de compatibilidad que no permitieron hacerlo funcionar. El backup se ejecuta manualmente con el script.

8.5 CORS en el backend

Al hacer peticiones desde el navegador al servidor Java surgían errores de CORS. El navegador bloqueaba las peticiones porque el frontend corría en `http://127.0.0.1:5500` y el backend en `http://localhost:8080`, considerándolos orígenes distintos. Se solucionó añadiendo la anotación `@CrossOrigin(origins = "*")` en la clase `usuarioControllerDAO`, permitiendo peticiones desde cualquier origen.

9. Conclusiones

El desarrollo de LoveCode ha permitido poner en practica de forma integrada los conocimientos de todos los módulos implicados: diseño de bases de datos relacionales con triggers y procedimientos almacenados, desarrollo de una API REST con Java y SpringBoot, creación de una interfaz web dinámica con JavaScript y control de versiones con Git.

9.1 Lo aprendido

- Diseño de bases de datos con tablas relacionadas, triggers y procedimientos almacenados.
- Desarrollo de APIs con Spring Boot y anotaciones `@RestController` y `@CrossOrigin`.
- Comunicación entre frontend y backend mediante `fetch()` y peticiones HTTP.
- Gestión de sesión en el navegador con `localStorage`.
- Control de versiones con Git, uso de ramas y pull requests en GitHub.
- Configuración de MariaDB en máquina virtual con acceso remoto.

10. Anexos

Base de datos

Backup creado

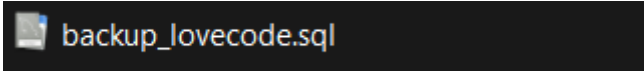
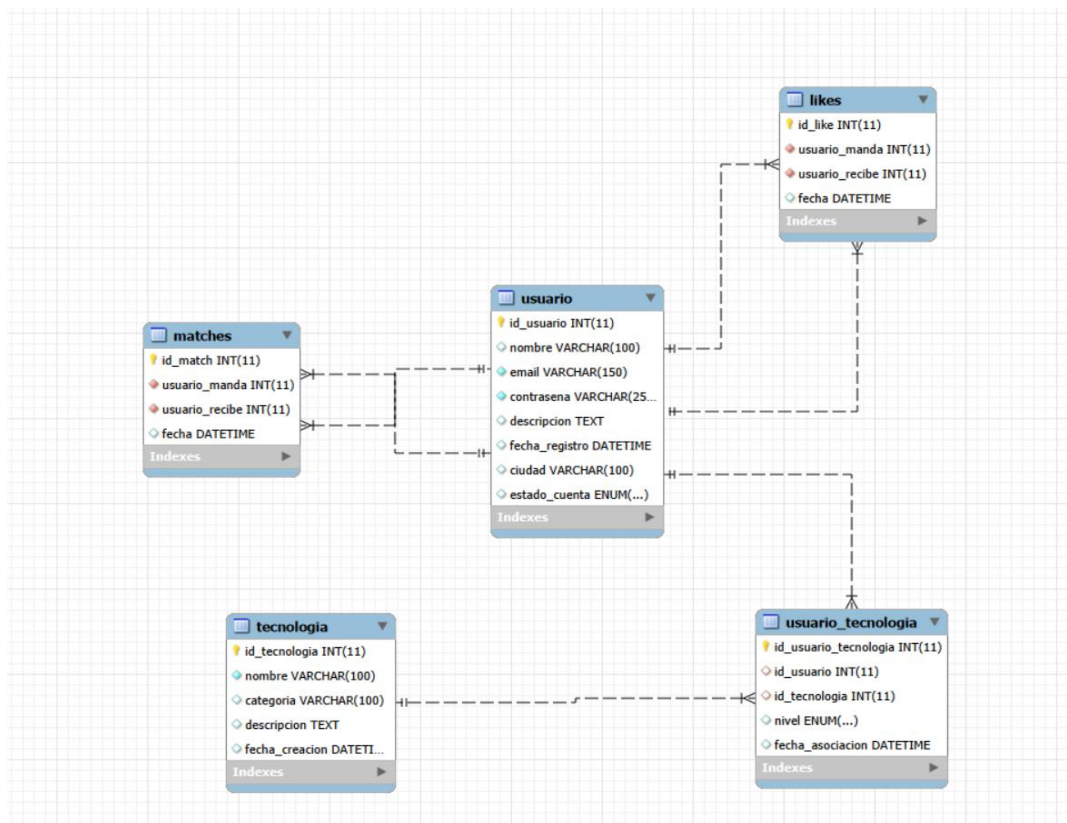


Tabla entidad relación

La base de datos tiene 5 tablas usuario es la tabla principal almacena los datos de cada usuario. Tecnología guarda las tecnologías disponibles en la plataforma se relaciona con usuario a través de la tabla intermedia usuario_tecnologia que indica las tecnologías que tiene cada usuario. La tabla likes registra cuando un usuario da like a otro guardando quien lo manda y quien lo recibe. Y matches se genera automáticamente mediante un trigger cuando dos usuarios se dan like mutuamente .



Scripts

Este script sirve para generar el backup de la base de datos

```
sergio@debian:~$ sudo mysqldump -u root -p LoveCode > backup_lovecode.sql
```

Y este sirve para mover el backup de la máquina virtual a la maquina anfitriona

```
sergio@debian:~$ cp backup_lovecode.sql /mnt/hgfs/dumps/
```